Project Arbitrage Module

SPEC PACK · V1.0.0

Ecosystem:	Quantum Workbook
Verticals:	CarHawk · Real Estate · ATV · Solar · Thrifty Mobile
System Version:	QUANTUM-2.1.0
Module Type:	Universal Add-On
Total Code:	2,936 lines · 8 files
Build Date:	2026-09-05

A · Module Overview

The **Project Arbitrage Module (PAM)** extends the Quantum workbook ecosystem with cross-vertical project needs matching against scraped listings. It plugs into the existing skeleton (Import → Staging → Master DB → Enrichment → Deal Engine → Verdict → CRM Sync → Dashboard → Settings) as a parallel layer sitting alongside the Deal Engine.

> **Purpose.** For a solo operator running multiple arbitrage verticals from Google Sheets, PAM answers a specific question: *"Does this scraped listing satisfy a project need I already have?"* — turning general marketplace scraping into targeted, project-driven acquisition.

Two-Layer Architecture

1. **Layer 1 — Logic Engine.** Keyword + rule-based matching against open needs. Scores 0–100 across six weighted criteria. Fast, deterministic, runs on every scraped row.
2. **Layer 2 — AI Evaluation.** OpenAI-powered strategy layer that only runs on matches above a configurable threshold. Returns fit verdict, compatibility insight, strategy, risk flags, and negotiation angle.

Cross-Project Detection

When one listing satisfies needs from multiple projects (e.g., a generator that fits both an off-grid solar build and a rental inventory need), PAM detects the secondary match and factors both into the AI evaluation.

Zero Breaking Changes

All PAM logic lives in 7 dedicated `.gs` files plus one HTML file. Existing CarHawk, Turo, and CRM modules are untouched.

B · File Inventory

#	FILE	TYPE	LINES	PURPOSE
1	`PAM_Settings.gs`	Apps Script	113	Config load/save, defaults, API key resolution
2	`PAM_Utils.gs`	Apps Script	152	ID generation, dedup, formatters, keyword scoring
3	`PAM_Sheets.gs`	Apps Script	220	Sheet creation, dropdowns, conditional formatting
4	`PAM_DataAccess.gs`	Apps Script	310	CRUD layer called by HTML UI via google.script.run
5	`PAM_LogicEngine.gs`	Apps Script	155	Layer 1 scoring engine — 6 weighted criteria
6	`PAM_AIEngine.gs`	Apps Script	260	Layer 2 OpenAI evaluator, prompt builder, JSON parser

#	FILE	TYPE	LINES	PURPOSE
7	`PAM_Menu.gs`	Apps Script	72	Menu registration, dashboard opener, test seeder
8	`PAM_UI.html`	HTML/CSS/JS	1,654	Full-screen dark command center (5 tabs)
Total			**2,936**	across 8 files

C · Sheet Architecture

Five sheets created programmatically. Idempotent — safe to re-run `initializePAMSheets()`.

#	SHEET	COLS	PURPOSE
1	`PAM_Projects`	13	Master list of active projects with budget tracking
2	`PAM_Needs`	19	Item-level needs linked to projects, with keyword sets
3	`PAM_Matches`	25	Engine output — scored listing×need pairs with AI verdicts
4	`PAM_Purchases`	15	Actual purchases, feeds budget tracking
5	`PAM_Dashboard`	query	Read-only command view with 5 sections
6	`PAM_Settings`	3	Configuration key/value store

D · Schema — PAM_Projects (13 cols)

COL	HEADER	TYPE	SOURCE	NOTES
A	Project ID	Text	Manual	Auto-suggested P001, P002…
B	Project Name	Text	Manual	e.g. "Berkeley House Rebuild"
C	Project Type	Dropdown	Manual	8 options — Vehicle Flip, Property Rehab, etc.
D	Category	Text	Manual	Free-form grouping
E	Description	Text	Manual	Brief scope
F	Budget	Currency	Manual	Total allocated
G	Spent So Far	Currency	**FORMULA**	`=SUMIFS(PAM_Purchases!F:F, PAM_Purchases!B:B, A2)`
H	Remaining Budget	Currency	**FORMULA**	`=F2-G2`
I	Priority	Dropdown	Manual	Critical, High, Medium, Low
J	Status	Dropdown	Manual	Planning, Active, Paused, Complete, Cancelled

COL	HEADER	TYPE	SOURCE	NOTES
K	Start Date	Date	Manual	
L	Target Completion	Date	Manual	
M	Notes	Text	Manual	

Conditional formatting: Row highlights red when Remaining Budget drops below 10% of Budget.

E · Schema — PAM_Needs (19 cols)

COL	HEADER	TYPE	SOURCE	NOTES
A	Need ID	Text	Manual	N001, N002…
B	Project ID	Text	Manual	FK → PAM_Projects.A
C	Project Name	Text	FORMULA	`=VLOOKUP(B2, PAM_Projects!A:B, 2, FALSE)`
D	Item Needed	Text	Manual	e.g. "Mini Split"
E	Keyword Set	Text	Manual	Comma-separated matching terms
F	Category	Dropdown	Manual	17 options
G	Subcategory	Text	Manual	Optional
H	Preferred Brand	Text	Manual	
I	Preferred Model	Text	Manual	
J	Acceptable Alternatives	Text	Manual	Fallback brands
K	Condition Needed	Dropdown	Manual	New, Like New, Good, Fair, Any
L	Qty Needed	Number	Manual	
M	Qty Acquired	Number	FORMULA	COUNTIFS from Purchases
N	Target Price	Currency	Manual	Ideal buy price
O	Max Price	Currency	Manual	Absolute ceiling
P	Priority	Dropdown	Manual	Critical, Important, Nice to Have
Q	Required	Checkbox	Manual	
R	Notes	Text	Manual	
S	Status	Dropdown	Manual	Open, Partially Filled, Fulfilled, Cancelled

F · Schema — PAM_Matches (25 cols)

Engine output. Only qualifying listings (score ≥ 40) are written.

COL	HEADER	TYPE	SOURCE
A	Match ID	Text	AUTO — M001, M002…
B	Scraped Row ID	Text	AUTO — Reference to Master DB
C	Project ID	Text	AUTO — From matching
D	Project Name	Text	FORMULA — VLOOKUP
E	Need ID	Text	AUTO — Primary matched need
F	Need Item	Text	FORMULA — VLOOKUP
G	Secondary Need ID	Text	AUTO — Cross-project match
H	Listing Title	Text	AUTO
I	Listing Description	Text	AUTO — Truncated 500 chars
J	Platform	Text	AUTO — Facebook, OfferUp, etc.
K	Listing URL	Text	AUTO
L	Asking Price	Currency	AUTO
M	Estimated Value	Currency	AUTO — From Deal Engine if present
N	Distance	Text	AUTO — If location data exists
O	Logic Match	Boolean	AUTO — Always TRUE for written rows
P	Match Score	Number	AUTO — 0–100 composite
Q	Match Tier	Text	AUTO — Weak / Good / Strong
R	AI Fit Verdict	Text	AI Perfect / Likely / Possible / Not Recommended
S	AI Compatibility Insight	Text	AI 2–3 sentences
T	AI Strategy	Text	AI Buy for Project / Buy + Flip / Watch / Pass
U	AI Risk Flags	Text	AI CSV of risks
V	AI Negotiation Angle	Text	AI Suggested offer approach
W	Recommended Action	Dropdown	AUTO/AI — Derived from AI strategy
X	Review Status	Dropdown	Manual — New, Reviewed, Acted On, Dismissed
Y	Timestamp	DateTime	AUTO — When match was generated

G · Schema — PAM_Purchases (15 cols)

COL	HEADER	TYPE	NOTES
A	Purchase ID	Text	PUR001…
B	Project ID	Text	FK → Projects
C	Need ID	Text	FK → Needs
D	Item Bought	Text	
E	Source / Platform	Text	
F	Purchase Price	Currency	Feeds project Spent-So-Far
G	Date Bought	Date	
H	Seller	Text	
I	Listing URL	Text	
J	Match ID	Text	Back-link to triggering match
K	Installed Yet	Checkbox	
L	Resellable Later	Checkbox	
M	Estimated Resale Value	Currency	If resellable
N	Condition	Dropdown	New, Like New, Good, Fair, Poor
O	Notes	Text	

H · Logic Engine (Layer 1) — Scoring Model

Entry point: `runPAMLogicEngine()`

Process

1. Loads `PAM_SourceSheet` (default: `Master DB`) rows.
2. Loads `PAM_Needs` where Status ∈ {Open, Partially Filled}.
3. For each listing × need pair, computes a weighted score.
4. Deduplicates by (Scraped Row ID, Need ID).
5. Writes qualifying rows (score ≥ 40) to `PAM_Matches`.
6. Detects cross-project secondary matches and populates Secondary Need ID.

Scoring — 100 pts possible

CRITERION	WEIGHT	CALCULATION
Keyword Match	40	`matched_keywords / total_keywords` — comma-split, case-insensitive, partial
Category Match	20	Listing category text-contains need category (bidirectional)
Price vs Target	20	Full pts if asking ≤ target; half pts if asking ≤ max
Brand/Model Bonus	10	Any preferred brand token (>2 chars) found in title+desc
Distance Score	5	Full pts if within radius; half if no distance data
Condition Clue	5	Listing text mentions matching/exceeding condition tier

Tier bands: 0–39 skip · 40–59 Weak · 60–79 Good · 80–100 Strong.

> All weights and thresholds live in `PAM_Settings`, never hardcoded.

I · AI Evaluation (Layer 2)

Entry point: `runPAMAIEvaluation()`
Model: `gpt-4o-mini` (configurable) — chosen for cost/latency balance.
Rate limit: 1.1 sec between calls · 20 rows per run default.

Trigger criteria: Match Score ≥ `PAM_AIThreshold` (default 60) AND `AI Fit Verdict` is empty OR starts with `"AI Error"`.

Prompt Structure

```
You are a deal evaluation assistant for a multi-vertical arbitrage operator.

PROJECT: {name} — {type}
NEED: {item}
PREFERRED: {brand} / {model}
CONDITION NEEDED: {condition}
TARGET PRICE: ${target}
MAX PRICE: ${max}
BUDGET REMAINING: ${remaining}
[SECONDARY PROJECT MATCH: ... if cross-project]

LISTING TITLE: {title}
LISTING DESCRIPTION: {desc}
ASKING PRICE: ${asking}
PLATFORM: {platform}
MATCH SCORE: {score}/100

Return JSON: {fit_verdict, compatibility_insight, strategy,
              risk_flags, negotiation_angle}
```

Response Contract

```
{
  "fit_verdict": "Perfect Fit | Likely Fit | Possible Fit | Not Recommended",
  "compatibility_insight": "2-3 sentence assessment",
  "strategy": "Buy for Project | Buy + Flip Extra | Watch | Pass",
  "risk_flags": "Comma-separated risks or 'None identified'",
  "negotiation_angle": "1-2 sentence approach + target offer"
}
```

Strategy → Action Mapping

AI STRATEGY	RECOMMENDED ACTION
Buy for Project	Buy for Project
Buy + Flip Extra	Buy for Project + Resale
Watch	Watch
Pass	Pass

Error handling: API failures write `"AI Error — retry"` to Fit Verdict; that row re-queues on the next run automatically.

J · HTML Command Center (5 Tabs)

Full-screen modal dialog, 1400×900. Dark theme, gold/blue accents, Playfair Display serif headings, monospace data.

#	TAB	CONTENTS
1	**Projects**	Stats chips · project row-cards with budget bars · expand-to-needs · inline New Project form
2	**Needs**	Filter bar · grouped by project (collapsible) · qty bars · inline Add Need form
3	**Matches**	Live stats bar · 4-filter bar · score cards with expandable AI detail · Buy/Watch/Pass action buttons · Purchase quick-add modal
4	**Run Engine**	3 pulse-hover action buttons (Logic / AI / Full Cycle) · live log console · current-settings preview · sheet initialize safety
5	**Settings**	Source sheet · 6 weight inputs with live-sum validator (must=100) · AI threshold slider with tier preview · AI model dropdown · save/reset

Design System

- **Palette:** `#0A0A0C` bg · `#C9A84C` gold · `#4080C4` blue · `#40A060` green · `#C48840` amber · `#C44040` red
- **Typography:** Playfair Display headings, Segoe UI body, SF Mono for data
- **Effects:** gold-glow card hover, pulse-animation on primary buttons, noise-texture overlay at 0.028 opacity
- **Security:** All user content passed through `esc()` XSS sanitiser

google.script.run Bindings

`getPAMProjects`, `addProject`, `getPAMNeeds`, `addNeed`, `getPAMMatches`, `updateMatchStatus`, `recordPurchase`, `runPAMLogicEngine`, `runPAMAIEvaluation`, `runPAMFullCycle`, `getPAMSettings`, `savePAMSettings`, `getPAMStats`, `getPAMNextIds`, `initializePAMSheets`

K · Settings Reference

Stored in `PAM_Settings` sheet, key/value rows.

SETTING	DEFAULT	DESCRIPTION
`PAM_SourceSheet`	`Master DB`	Sheet name for scraped listings
`PAM_KeywordWeight`	`40`	Keyword match points
`PAM_CategoryWeight`	`20`	Category match points
`PAM_PriceWeight`	`20`	Price vs target points

SETTING	DEFAULT	DESCRIPTION
`PAM_BrandWeight`	10	Brand/model bonus points
`PAM_DistanceWeight`	5	Distance points
`PAM_ConditionWeight`	5	Condition clue points
`PAM_AIThreshold`	60	Min score to trigger AI
`PAM_AIModel`	gpt-4o-mini	OpenAI model
`PAM_MaxAIPerRun`	20	Max API calls per run
`PAM_DistanceRadius`	30	Miles
`PAM_APIKeyCell`	Settings!B2	Cell ref for OpenAI key (never stored in PAM)

L · Menu & Entry Points

```
⚡ PAM
├── Open Command Center          → openPAMDashboard()
├── ──────────
├── Run Logic Engine             → runPAMLogicEngine()
├── Run AI Evaluation            → runPAMAIEvaluation()
├── Full Cycle (Logic → AI)      → runPAMFullCycle()
├── ──────────
└── Initialize PAM Sheets        → initializePAMSheets()
```

Registered via `addPAMMenu()` — called from the existing `createQuantumMenu()` in `quantum_menu.gs`.

M · Deployment Checklist

1. Copy all 7 `.gs` files into the Apps Script project.
2. Copy `PAM_UI.html` as an HTML file (name must be `PAM_UI` — matches `HtmlService.createHtmlOutputFromFile('PAM_UI')`).
3. Add `addPAMMenu()` call inside `createQuantumMenu()` in `quantum_menu.gs`.
4. Verify OpenAI API key exists at the cell referenced by `PAM_APIKeyCell` (default `Settings!B2`).
5. Reload the workbook — menu appears.
6. Run ⚡ **PAM** → **Initialize PAM Sheets** — creates all 5 sheets + Settings rows.
7. Optional: run `seedPAMTestData()` from Apps Script editor for 3 test projects + 7 needs.
8. Populate real projects/needs via the Command Center UI.
9. Run ⚡ **PAM** → **Full Cycle** — Logic Engine scores all Master DB rows, AI evaluates qualifying matches.

10. Review the Matches tab; use action buttons to record purchases or dismiss.

N · Test Data (seedPAMTestData)

ID	PROJECT	TYPE	BUDGET
P001	Berkeley House Rebuild	Property Rehab	$15,000
P002	Yamaha Blaster Build	ATV/Powersports Build	$2,500
P003	CarHawk Flip 04 — Mustang	Vehicle Flip	$4,000

ID	NEED	PROJECT	TARGET / MAX	PRIORITY
N001	Mini Split AC Unit	P001	$250 / $500	CRITICAL
N002	Exterior Entry Door	P001	$100 / $250	IMPORTANT
N003	Bathroom Vanity	P001	$75 / $200	NICE
N004	Front Plastics Set	P002	$40 / $80	CRITICAL
N005	Brake Setup	P002	$30 / $60	IMPORTANT
N006	Headlight Assembly	P003	$80 / $150	CRITICAL
N007	Front Seat Set	P003	$150 / $300	IMPORTANT

O · Critical Rules (Non-Negotiable)

1. Spent So Far and Remaining Budget on Projects are **always formulas** pulling from Purchases. Never manual.
2. Project Name on Needs, Matches, and Purchases is **always a VLOOKUP** from Projects. Never typed.
3. Scoring weights live in **Settings**. Never hardcoded in logic functions.
4. AI **only** runs on matches above threshold. Never on every row.
5. Deduplication: **never** create duplicate matches for the same (Scraped Row ID, Need ID) pair.
6. The HTML UI is the **primary interface**. Sheets are the data layer.
7. All currency values use `$` formatting. All IDs auto-increment.
8. Idempotency: `initializePAMSheets()` is safe to run repeatedly — never corrupts data.
9. Error handling everywhere — API failures, missing sheets, empty data, malformed responses.
10. The HTML is a **command center**, not a form. Dark, luxurious, operationally clear.